



BugIt
by Taskivator

Manuel complet, étape par étape

BugIt

Agent QA de création de bugs pour VS Code

Configuration initiale et utilisation quotidienne : un guide détaillé, clic par clic, écrit pour des personnes qui n'ont jamais fait ça. Aucune compétence technique requise. Suivez-le dans l'ordre et vous créerez des tickets impeccables dès aujourd'hui.

Fonctionne dans : VS Code sur Windows (principal), macOS & Linux

Avertissement sur la traduction.

Ce document a été traduit automatiquement et n'a pas été relu par des locuteurs natifs. La version anglaise fait foi : en cas de divergence, le texte anglais prévaut. Pour la formulation la plus exacte et la plus à jour, veuillez vous reporter au document en anglais.

CONTENU DU GUIDE

Faites la **Partie 1** une fois, dans l'ordre. Tout ce qui suit peut être consulté au besoin. Le **Dépannage** et la **FAQ** en fin de guide répondent aux questions les plus fréquentes, alors merci de les consulter avant d'écrire au support.

Partie 1 · Configuration (à faire une fois)

Étape 0 : Obtenir GitHub Copilot

Étape 1 : Installer VS Code

Étape 2 : Décompresser votre téléchargement Buglt

Étape 3 : Ouvrir Buglt dans VS Code

Étape 4 : Choisir l'agent Buglt dans le chat

Étape 5 : Activer votre licence

Étape 6 : Accepter les conditions (une fois)

Étape 7 : Lancer « Begin setup »

Étape 8 : Connecter votre outil de suivi

Étape 9 : Vérifier vos identifiants enregistrés

Étape 10 : Confirmer que vous êtes prêt

Partie 2 · Utilisation au quotidien

Partie 3 · Personnalisation

Partie 4 · Mises à jour, sauvegardes et sécurité

Partie 5 · Dépannage et FAQ

Partie 6 · Licence et assistance

La seule règle qui vous protège

Buglt ne crée **jamais** rien sans vous montrer un aperçu et obtenir une confirmation tapée au clavier : les actions irréversibles exigent de saisir **FILE IT**, car un simple « oui » ne suffit pas. Vous gardez toujours le contrôle. Envie de vous entraîner sans aucun risque ? Tapez **dry run** et il rédige le rapport complet **sans** le créer.

Configuration initiale vs. utilisation au quotidien

Vous ne suivez toute la **Partie 1** qu'une seule fois. Ensuite, ouvrir Buglt est rapide, et les mises à jour tiennent en une commande (**Update**). Voir la Partie 4.

PARTIE 1 · CONFIGURATION : À FAIRE UNE FOIS

Environ 15 minutes. Suivez les étapes dans l'ordre ; chacune s'appuie sur la précédente. Ne sautez pas d'étape.

Obtenir GitHub Copilot

Ce guide suit le parcours GitHub Copilot

BugIt est le « conducteur » et l'IA est le « moteur » (c'est elle qui rédige réellement les rapports). Ces étapes vous installent avec GitHub Copilot dans VS Code, le chemin le plus rapide si vous partez de zéro. BugIt fonctionne aussi avec l'extension Claude, avec tout autre assistant capable d'exécuter une commande, et seul avec votre propre clé d'IA. Si vous utilisez déjà l'un d'eux, passez à l'Étape 2 et ouvrez le dossier BugIt dans cet outil plutôt que dans VS Code.

De quoi s'agit-il : GitHub Copilot est un assistant IA de GitHub (propriété de Microsoft). Il propose un palier gratuit pour un usage léger et un palier payant pour un usage intensif. Vous l'installerez à l'Étape 1 et vous vous y connecterez.

1. Il vous faut un **compte GitHub**. Si vous n'en avez pas, créez-en un gratuitement sur github.com/signup.
2. Le palier gratuit de Copilot suffit pour essayer BugIt. Si vous créez des bugs toute la journée, l'offre payante « Copilot Pro » en vaut la peine, et vous pouvez passer à niveau à tout moment sur github.com/features/copilot.
3. Vous activerez réellement Copilot dans VS Code à l'Étape 1, donc inutile de faire quoi que ce soit d'autre ici pour l'instant.

Installer VS Code

VS Code est une application gratuite de Microsoft. C'est la « fenêtre » dans laquelle vit BugIt.

1. Ouvrez votre navigateur web et allez sur code.visualstudio.com.
2. Cliquez sur le gros bouton bleu **Download** (il détecte automatiquement votre système).
3. Ouvrez le fichier téléchargé et installez-le : **acceptez toutes les options par défaut**, contentez-vous de cliquer sur Suivant/Installer.
4. Ouvrez VS Code une fois l'installation terminée.

Activer GitHub Copilot dans VS Code

1. Tout à gauche de VS Code, cliquez sur l'icône **Extensions** (elle ressemble à quatre petits carrés).
2. Dans la zone de recherche, tapez **GitHub Copilot**.
3. Cliquez sur **Install** pour « GitHub Copilot » (et, si proposé, « GitHub Copilot Chat »).
4. Une invite vous demandera de **Sign in to GitHub**. Cliquez dessus et connectez-vous avec votre compte GitHub dans la fenêtre de navigateur qui s'ouvre.

✓ **Vous saurez que ça a fonctionné quand :** une petite icône Copilot apparaît en bas de VS Code, et le panneau de chat s'ouvre (l'icône en forme de bulle à gauche, ou **Ctrl+Alt+I**).

2

Décompresser votre téléchargement BugIt

Votre achat est accompagné d'un fichier .zip (par exemple `bugit-qa-agent.zip`). Vous devez le décompresser ; l'exécuter compressé ne fonctionnera pas.

1. Trouvez le fichier .zip (généralement dans votre dossier **Téléchargements**).
2. **Clic droit** dessus → choisissez **Extract All...** (Windows) ou double-cliquez dessus (Mac).
3. Choisissez un emplacement simple comme votre dossier **Documents**, puis cliquez sur Extract.
4. Vous avez maintenant un dossier **BugIt** contenant de nombreux fichiers. Repérez son emplacement.

Ne le placez pas dans OneDrive ou un dossier synchronisé

Les dossiers synchronisés dans le cloud peuvent provoquer des conflits de fichiers. Votre dossier Documents ou Bureau convient parfaitement. Ne renommez pas non plus les fichiers à l'intérieur ; laissez simplement le dossier tel quel.

3

Ouvrir BugIt dans VS Code

1. Dans VS Code, cliquez sur **File → Open Folder** (menu en haut à gauche).
2. Naviguez jusqu'à l'endroit où vous avez décompressé BugIt (par ex. Documents).
3. Cliquez une fois sur le dossier **BugIt** pour le sélectionner, puis cliquez sur **Select Folder**.
4. Si VS Code demande « Do you trust the authors of the files in this folder? », cliquez sur **Yes, I trust the authors**.

✓ **Vous saurez que ça a fonctionné quand** : vous voyez les fichiers BugIt (des dossiers comme `tools` , `.github` , et des fichiers comme `README.md`) listés sur la gauche de VS Code.

4

Choisir l'agent BugIt dans le chat

1. Ouvrez le panneau **Chat** de Copilot : cliquez sur l'icône en forme de bulle à gauche, ou appuyez sur `Ctrl+Alt+I` (Windows) / `Cmd+Alt+I` (Mac).
2. En haut de la zone de chat se trouve un petit **menu déroulant de mode** (il peut afficher « Ask » ou « Agent »).
3. Cliquez dessus et choisissez **BugIt QA Agent** dans la liste.

Vous ne voyez pas « BugIt QA Agent » dans la liste ?

Vérifiez que vous avez bien ouvert le dossier BugIt (Étape 3), et non un autre dossier ; l'agent se charge depuis l'intérieur de celui-ci. Fermez et rouvrez le dossier si nécessaire.

5

Activer votre licence

Un compte Buglt suffit ; il n'y a aucun code de licence à copier ni à coller. L'activation se déroule dans votre navigateur.

1. Dans la zone de chat, saisissez **hi** et appuyez sur Entrée.
2. Saisissez **Activate**. Buglt ouvre le Buglt Portal dans votre navigateur. Connectez-vous à votre propre compte, choisissez votre droit Solo ou Team, puis approuvez cet appareil.
3. Revenez dans VS Code et Buglt termine l'autorisation automatiquement ; votre mot de passe reste dans le navigateur et n'est jamais saisi dans VS Code.

✓ **Vous saurez que ça a fonctionné quand** : Buglt confirme l'activation et nomme le compte avec lequel vous êtes connecté.

Gardez votre compte confidentiel

Votre compte et votre droit vous sont propres. Ne les partagez pas, ne les publiez pas, ne les revendez pas ; un accès partagé peut être révoqué. Les membres d'une équipe se connectent chacun avec leur propre compte : il n'existe aucun code partagé ni identifiant de connexion partagé. Changer d'ordinateur plus tard ne pose aucun problème (voir la FAQ).

6

Accepter les conditions (une fois)

La première fois, Buglt affiche un court résumé et vous demande de l'accepter avant de faire quoi que ce soit d'autre.

1. Lisez le court résumé affiché (vous relisez chaque ticket avant qu'il ne soit créé ; la sécurité de votre projet vous incombe ; les protections sont des aides, pas des garanties, et comme elles sont suivies par le modèle d'IA qui vous répond dans Copilot, un modèle plus léger/gratuit peut occasionnellement sauter une étape ou avoir besoin qu'on répète une commande, là où un modèle plus puissant ne le ferait pas).
2. Choisissez **I agree** parmi les deux options proposées par Buglt.

✓ **Vous saurez que ça a fonctionné quand** : Buglt confirme et poursuit la configuration. Cette question ne vous est posée qu'une fois ; il s'en souvient.

7

Lancer « Begin setup »

Buglt se configure maintenant lui-même en vous interrogeant en langage courant. Vous ne modifiez jamais un fichier de paramètres à la main.

VOUS TAPEZ

Begin setup

Il posera de courtes questions et n'activera que ce que vous choisirez. La plus importante concerne votre outil de suivi.

8

Connecter votre outil de suivi

La configuration a enregistré *quel* outil de suivi vous utilisez. Cette étape est celle qui permet à BugIt d'y créer réellement les tickets : vous créez un jeton d'API dans votre propre compte, puis vous l'enregistrez sur cet ordinateur avec une seule commande. BugIt crée alors les tickets directement par l'API REST de votre outil de suivi avec ce jeton : rien n'a besoin de rester actif, et la création de tickets n'utilise jamais de serveur MCP.

8a · Créer un jeton dans votre propre compte

VOTRE OUTIL DE SUIVI	OÙ LE CRÉER
Jira Cloud	id.atlassian.com/manage-profile/security/api-tokens
Azure DevOps	dev.azure.com/YOUR-ORG/_usersSettings/tokens
GitHub Issues	github.com/settings/personal-access-tokens/new
GitLab Issues	gitlab.com/-/user_settings/personal_access_tokens
Bugzilla	dans votre propre Bugzilla : Preferences → API Keys
YouTrack	dans votre propre YouTrack : avatar → Profile → Account Security → Authentication
Linear	linear.app/settings/account/security
Shortcut	app.shortcut.com/settings/account/api-tokens
ClickUp	app.clickup.com/settings/apps
Asana	app.asana.com/0/my-apps
Trello	trello.com/apps/admin (une clé d'API ET un jeton que vous autorisez avec elle)

Copiez votre jeton dès qu'il s'affiche

La plupart des services n'affichent un nouveau jeton qu'une seule fois. Copiez-le immédiatement et conservez-le en lieu sûr jusqu'à ce que BugIt l'enregistre pour vous (Étape 9). Choisissez la durée de validité la plus longue proposée par votre outil de suivi pour ne pas avoir à recommencer trop vite.

8b · L'enregistrer sur cet ordinateur

Ouvrez un terminal dans VS Code et lancez la commande de votre outil de suivi, en mettant `ado`, `github`, `gitlab` et ainsi de suite à la place de `jira`.

VOUS EXÉCUTEZ (DANS LE TERMINAL)

```
python tools/connect.py jira
```

8c · Coller le jeton dans l'invite masquée

La commande demande le jeton dans une invite locale masquée : il n'apparaît jamais à l'écran et n'atteint jamais le chat, un ticket ni `config.json`. Collez-le là et nulle part ailleurs. BugIt se connecte ensuite, liste les projets que le jeton voit réellement, lit les types de tickets que vous avez le droit de créer, et **refuse d'enregistrer un identifiant incapable de créer un ticket**. Une configuration qui se termine signifie donc que la création fonctionne.

8d · Vérifier que la connexion fonctionne

Pour le confirmer à tout moment, lancez la vérification dans le terminal, ou demandez simplement dans le chat :

VOUS EXÉCUTEZ (DANS LE TERMINAL)

```
python tools/connect.py jira --check
```

VOUS TAPEZ

```
Check status
```

✓ **Vous saurez que ça a fonctionné quand** : BugIt indique que votre outil de suivi est connecté / en bonne santé.

Rien ne doit rester actif

Votre jeton d'API se trouve dans le coffre d'identifiants de votre système d'exploitation, pas dans VS Code. Il n'y a donc rien à démarrer quand vous rouvrez VS Code : BugIt peut créer des tickets immédiatement. Vous pouvez le vérifier à tout moment avec `python tools/connect.py jira --check`.

9

Vérifier vos identifiants enregistrés

Quand vous enregistrez votre jeton (Étape 8c), l'authentification est conservée dans le coffre d'identifiants de votre système d'exploitation (Windows Credential Manager / macOS Keychain / Linux Secret Service), jamais dans un fichier en clair et jamais envoyée où que ce soit. Vous n'avez rien à ressaisir à chaque fois.

VOUS TAPEZ

```
Check saved credentials
```

Cela indique **quels outils de suivi ont un identifiant enregistré**, jamais les valeurs des identifiants. BugIt n'affiche ni ne copie jamais la valeur d'un jeton ; les secrets restent dans le coffre de votre système.

Un identifiant a cessé de fonctionner ?

Relancez `python tools/connect.py jira` et collez un jeton neuf : il remplace l'ancien. BugIt n'affiche jamais les valeurs des identifiants enregistrés.

10 Confirmer que vous êtes prêt

Demandez à BugIt de tout revérifier avant de créer un vrai ticket :

VOUS TAPEZ

`Check readiness`

Il liste ce qui manque encore (généralement juste « connecter votre outil de suivi » ou « activer »). Quand tout est au vert, vous verrez « 🟢 Setup complete. »

La configuration est terminée, pour de bon

Vous ne referez pas la Partie 1. Désormais, ouvrir BugIt se résume à : ouvrir le dossier → taper `hi` . Pour changer un choix plus tard, tapez simplement `Begin setup` et indiquez ce qu'il faut changer.

Pas de GitHub Copilot ? Il existe un assistant en terminal

Si vous ne pouvez pas utiliser Copilot, ouvrez le terminal intégré (Terminal → New Terminal) et lancez `python tools/setup_wizard.py` . Il pose quelques questions et écrit vos paramètres sans l'IA. Vous pouvez aussi exécuter BugIt en mode autonome avec votre propre clé d'IA ; voir la FAQ.

PARTIE 2 · UTILISATION AU QUOTIDIEN

Une fois configuré, vous travaillez entièrement depuis le chat. Parlez naturellement, ou utilisez les exemples précis ci-dessous. Tapez `help` à tout moment pour la liste complète.

Rédiger un rapport de bug

La mission principale de BugIt. Décrivez le bug en une phrase simple ; il rédige le ticket complet, remplit automatiquement la version et la catégorie, vérifie les doublons, vous montre un aperçu, et le crée après que vous avez confirmé en saisissant `FILE IT` .

VOUS TAPEZ

`Write a bug report: the app crashes every time I tap Save on iPhone`

Il rédige un titre, les étapes de reproduction, le résultat attendu vs. constaté, la gravité et la plateforme, et il définit toujours le champ de priorité/gravité propre à l'outil de suivi pour qu'il corresponde, pas juste une valeur générique par défaut. Des changements avant la création ? Dites-le simplement, par exemple « make the severity high » ou « add that it started in the latest build ».

Bugs rapides (création accélérée)

Pour les cas courants (crash, error, missing, layout, slow, stuck, data, visual, audio, blocker), le formulaire rapide saute certaines questions et remplit la catégorie et la priorité pour vous, et saute la recherche de doublons pour éviter l'aller-retour. Utilisez-le lorsque vous savez déjà que le problème est nouveau ; l'aperçu vous le dit, et un rapport standard effectue toujours la vérification complète des doublons.

VOUS TAPEZ (EXEMPLES)

```
Quick bug: crash on startup after update
Quick bug: layout button overlaps text on the settings screen
Quick bug: blocker can't continue past the payment step
Quick bug: slow the dashboard takes 20 seconds to load
```

Rapports de bug de crash

Si vous avez connecté un outil de crash (comme Sentry), BugIt fait tout ce qui précède en plus d'associer votre rapport au crash et d'en récupérer la pile d'appels pour vous.

VOUS TAPEZ

```
Write a crash bug report: game crashes when opening the map
```

Commentaires de vérification et de clôture

Après avoir testé un correctif, BugIt rédige un commentaire soigné à publier sur le ticket. Il rédige (et publie éventuellement) le commentaire ; il ne change pas le statut du ticket. Vous clôturez/résolvez/rouvrez toujours le ticket vous-même dans votre outil de suivi.

VOUS TAPEZ	CE QUE VOUS OBTENEZ
Close #123	Demande quel scénario (correctif confirmé, clôturé sur demande, comportement voulu, réouverture...) puis rédige ce commentaire.
Write a verification comment for the build you tested on Windows	Un commentaire « correctif confirmé » avec les détails de la version renseignés.
Write a reopen comment for the build you tested on Windows	Un commentaire « le problème persiste » (vous rouvrez ensuite le ticket vous-même).

Traduction

Traduisez des rapports ou des termes entre vos langues configurées, en utilisant la formulation exacte de votre équipe (issue de votre glossaire ; voir la Partie 3).

VOUS TAPEZ (EXEMPLES)

```
Translate to ja: the save button does nothing on the profile screen
Translate this bug report to English: [paste text]
```

Rechercher des informations (specs et glossaire)

Si vous avez connecté vos specs ou rempli votre glossaire, posez des questions à BugIt sur votre projet.

VOUS TAPEZ (EXEMPLES)

What is [your term]?
Walk me through the checkout flow
What's new in specs?

Détection des doublons

Cela se produit automatiquement pour un rapport standard, avant que quoi que ce soit ne soit écrit : BugIt recherche dans votre outil de suivi des tickets correspondants (même formulés un peu différemment) et vous avertit, pour que vous ne créiez jamais le même bug deux fois. Le formulaire rapide saute volontairement la recherche pour éviter l'aller-retour, et le dit dans l'aperçu, afin que vous sachiez toujours lequel des deux vous avez eu.

Fonctionnalités avancées (si vous les avez activées)

TAPEZ CECI	CE QUE ÇA FAIT
Triage digest	Standup instantané : bugs ouverts par domaine/gravité, les plus anciens signalés.
Export bug to file	Enregistre le rapport en Markdown/CSV/JSON au lieu de le créer (idéal avant que votre outil de suivi soit configuré).
Undo last filing	Affiche la dernière écriture du journal d'audit. BugIt ne peut ni supprimer ni clôturer un ticket ; après un nouvel aperçu et FILE IT , il peut seulement retirer un commentaire ou une pièce jointe qu'il a créés.
(automatique)	Alertes SLA sur les bugs urgents · champs spécifiques par catégorie.

La liste complète des commandes

VOUS TAPEZ

help

...affiche toutes les commandes de l'agent, à tout moment.

Entraînez-vous en toute sécurité, à tout moment

Tapez **dry run** et BugIt rédige le rapport complet **sans** créer quoi que ce soit, parfait pour apprendre ou tester.

PARTIE 3 · PERSONNALISATION : AJOUTEZ LES CONNAISSANCES DE VOTRE PROJET

Par défaut, Buglt est un agent QA vierge. Voici comment lui enseigner votre projet après l'achat. Aucun code requis : presque tout se fait en lui parlant dans le chat. Tout ce que vous ajoutez reste sur votre ordinateur.

1 · Les mots de votre équipe (le glossaire)

Cela rend les traductions et la détection de catégorie précises pour votre projet.

1. Dans la liste de fichiers, ouvrez [.github/glossary/terms.template.md](https://github.com/glossary/terms.template.md).
2. Remplissez le tableau avec les mots de votre équipe : noms de fonctionnalités, noms d'écrans, noms d'éléments, abréviations, et (si vous utilisez deux langues) leurs traductions.
3. Enregistrez le fichier (**Ctrl+S**). C'est tout ; Buglt utilise désormais ces termes exacts.

Raccourci

Activez « glossary auto-seed » pendant **Begin setup** et Buglt suggère des termes à partir de vos tickets existants, et vous n'avez plus qu'à approuver chacun d'eux.

2 · Votre style de bug (style maison)

Vous voulez que les tickets correspondent exactement au format de votre équipe : style de titre, libellés de gravité, champs obligatoires ? Vous ne le décrivez pas ; vous le **montrez** :

1. Dans le chat, dites : « match my team's style. »
2. Collez **une capture d'écran** d'un ticket existant de votre outil de suivi.
3. Buglt l'étudie (en local, en supprimant d'abord toute donnée personnelle) et enregistre votre format, qu'il applique ensuite à chaque futur ticket.

3 · Vos catégories, plateformes et champs

Dites-le simplement à Buglt en langage courant et il met à jour vos paramètres pour vous :

VOUS TAPEZ (EXEMPLES)

```
My categories are Gameplay, UI, Audio, and Networking
We test on Windows and Xbox
```

4 · Vos propres outils (connexions personnalisées)

Vous utilisez un outil absent de la liste intégrée ? Connectez-le exactement comme un outil intégré :

VOUS TAPEZ

```
Add a custom MCP server
```

Donnez-lui un nom court et son adresse (doit être en <https://>) ; Buglt le connecte et vérifie son état de santé pour vous.

5 · Corrigez-le simplement au fil de l'eau

La méthode la plus simple de toutes : quand BugIt rédige un ticket, corrigez tout ce qui ne vous plaît pas, un libellé, la formulation, la gravité. Avec « learn from your edits » activé (recommandé), il retient votre préférence et l'applique la fois suivante. Les connaissances de votre projet s'accumulent naturellement, et rien de tout cela ne quitte jamais votre machine.

Tout ce que vous ajoutez vous appartient et reste local

Votre glossaire, votre style maison, vos catégories et vos préférences apprises vivent sur votre ordinateur, sont sauvegardés avant chaque mise à jour, et ne sont jamais envoyés à qui que ce soit.

6 · Partager votre configuration avec votre équipe (Licence Équipe)

Une fois votre glossaire, votre style maison et vos choix d'outil de suivi configurés comme vous le souhaitez, donnez la même configuration exacte au reste de votre équipe en une seule commande, au lieu de répéter les étapes 1 à 3 sur chaque machine :

VOUS EXÉCUTEZ (UNE FOIS, QUAND VOTRE CONFIGURATION VOUS CONVIENT)

```
python tools/share.py export
```

Cela écrit `config.share.zip` : vos choix d'outil de suivi/de crash/de communication, votre glossaire et votre style maison, regroupés sans aucun mot de passe ni jeton à l'intérieur. Envoyez-le à votre équipe.

CHAQUE MEMBRE DE L'ÉQUIPE EXÉCUTE

```
python tools/share.py import config.share.zip
```

Ils obtiennent immédiatement votre terminologie exacte, votre style de bug et votre configuration d'outil de suivi, sans retaper les catégories et sans recoller de capture d'écran pour le style maison. Leur propre licence et les paramètres de leur appareil restent inchangés.

Si l'import modifie où les données sont envoyées

Si une configuration partagée ajoute une nouvelle connexion personnalisée ou une nouvelle adresse de notification, BugIt affiche exactement ce qui a changé avant que quoi que ce soit d'autre ne se produise ; ce n'est jamais appliqué silencieusement. Il capture aussi d'abord un instantané de la configuration actuelle du coéquipier, si bien que `Restore my settings` annule l'import si ce n'est pas ce à quoi il s'attendait.

PARTIE 4 · MISES À JOUR, SAUVEGARDES ET SÉCURITÉ

Obtenir les mises à jour (une commande)

Quand une nouvelle version est disponible, Buglt le mentionne une fois quand vous dites `hi`. Les mises à jour du logiciel sont incluses tant que votre licence Buglt est active. Pour l'installer :

VOUS TAPEZ

Update

1. Buglt effectue d'abord une sauvegarde fraîche de vos fichiers.
2. Il télécharge la nouvelle version et vérifie qu'elle est authentique et non altérée (elle est signée cryptographiquement, donc une mise à jour falsifiée ou altérée est refusée).
3. Il l'installe sans toucher à vos paramètres, votre glossaire, votre style maison, votre licence ou vos jetons.
4. Il vous demande de recharger VS Code (`Ctrl+Shift+P` → tapez « Reload Window » → Entrée). Démarrez un nouveau chat et vous êtes sur la nouvelle version.

Jamais besoin de supprimer un dossier

Contrairement à la première installation, les mises à jour se font sur place. Vous ne supprimez ni ne retéléchargez jamais rien, et votre configuration est toujours préservée et sauvegardée.

Ce qu'il est sûr de modifier à la main, et ce qui ne l'est pas

Sûr, et conservé à chaque mise à jour :

`config.json`, `.github/instructions/house-style.instructions.md` (voir la Partie 3 ; c'est la méthode prise en charge pour personnaliser le comportement), `.github/glossary/terms.template.md`, `.github/instructions/learned.instructions.md`, et `.vscode/mcp.json`.

Pas sûr : ne modifiez pas ces fichiers à la main

Le fichier de l'agent (`.github/agents/bugit-qa-agent.agent.md`), tout autre fichier sous `.github/instructions/`, et tout ce qui se trouve dans `tools/` sont le produit lui-même, pas vos paramètres. Une mise à jour les **écrase silencieusement**, et, contrairement aux fichiers ci-dessus, **il n'existe aucune sauvegarde permettant de les restaurer**. Buglt vérifie cela au démarrage puis juste avant d'installer une mise à jour, et vous avertit s'il détecte un changement, mais la règle la plus sûre reste simplement de ne pas les modifier.

Sauvegardes et restauration

VOUS TAPEZ

CE QUE ÇA FAIT

`Back up my settings`

Enregistre un instantané local de votre configuration, votre glossaire, votre style maison, vos endpoints MCP et vos préférences apprises. Votre droit d'appareil signé et les valeurs des identifiants sont exclus.

List backups Affiche chaque instantané enregistré avec sa date.

Restore my settings Revient à un instantané (et capture d'abord votre état actuel, si bien qu'une restauration est elle-même annulable).

Automatique avant chaque mise à jour

Buglt effectue toujours une sauvegarde avant d'installer une nouvelle version, si bien qu'une mise à jour ne peut jamais faire perdre votre configuration. Si quelque chose semble anormal, **Restore my settings** répare la situation.

Sécurité : ce qui est protégé

✓ Rien sans votre confirmation

Chaque ticket et commentaire est présenté en aperçu ; les actions irréversibles exigent de saisir **FILE IT**, car un simple « oui » ne suffit pas.

🧪 Dry-run

Dites **dry run** (ou **QA_AGENT_DRY_RUN=1**) : tout ticket, commentaire, modification, pièce jointe, suppression et notification externes sont supprimés (les commandes locales que vous lancez vous-même, comme la sauvegarde ou l'export, peuvent encore créer des fichiers locaux) et votre outil de suivi n'est pas contacté du tout, lectures comprises : pas de recherche, pas de vérification des doublons, pas de test de connexion, car chacun atteint aussi votre tableau et déverrouille votre jeton enregistré. **QA_AGENT_DRY_RUN=1** dans votre terminal est l'interrupteur que les commandes fournies appliquent dans le code ; dire « dry run » dans le chat demande à l'assistant de s'abstenir, ce qui est utile mais n'offre pas la même garantie.

🔍 Aucun secret dans les fichiers

Les jetons résident dans le coffre de votre système, jamais dans un fichier en clair, jamais envoyés à nous.

🔒 S'exécute sur votre machine

Il ne parle qu'aux outils que vous connectez et à votre propre modèle d'IA.

Vos données et votre confidentialité

La seule chose que Buglt envoie à Taskivator, ce sont des données de licence/mise à jour : la connexion à votre compte Buglt (dans le navigateur, sur le Portal), un identifiant d'appareil haché à sens unique, un identifiant d'installation aléatoire, le nom de votre appareil (nom d'hôte) et le nom de votre système d'exploitation, la version de Buglt et le filtre d'offre que vous choisissez, du matériel cryptographique de défi à durée de vie courte, et le droit Solo ou Team que vous approuvez pour cet appareil. Il n'y a aucun code de licence, et votre mot de passe reste dans le navigateur et n'est jamais saisi dans VS Code. Vos rapports de bug, vos specs, votre glossaire, vos paramètres et vos jetons ne quittent jamais votre machine. Aucune télémétrie, aucune analyse, jamais. Tous les détails figurent dans le fichier **PRIVACY.md** de votre dossier Buglt.

Risques et précautions importantes : à lire

L'IA peut se tromper

Buglt rédige les rapports avec un modèle d'IA, qui peut mal interpréter des détails ou en inventer. Lisez toujours l'aperçu avant de confirmer. C'est vous qui approuvez ce qui est créé.

Il crée le ticket dans votre outil de suivi réel

Une fois que vous approuvez, un vrai ticket est créé. Vous vous entraînez ? Utilisez `dry run` pour rédiger le rapport sans le créer.

La sécurité de votre projet vous appartient

La façon dont vous utilisez Buglt, ainsi que la sécurité de vos projets, données et outils de suivi, sont de votre responsabilité. Les protections réduisent le risque mais ne remplacent pas votre relecture.

Ce que Buglt ne fait pas

Buglt crée des tickets dans les onze outils qu'il nomme, chacun avec un identifiant que vous créez dans votre propre compte et que Buglt vérifie face à la destination choisie avant de l'enregistrer. Ce qui atteint un champ de priorité triable varie selon l'outil, et GitHub, GitLab et Trello n'en ont aucun : la gravité y est écrite dans le texte du ticket. Confluence, Sentry et Notion sont en lecture seule ; tout le reste passe par votre propre serveur MCP compatible, que vous configurez avec l'aide de Buglt. Il utilise votre IA (Copilot ou votre propre clé OpenAI/Anthropic), pas un modèle fourni ; le caviardage est fait au mieux ; et il fonctionne dans VS Code, dans l'extension Claude et dans un terminal ordinaire. Une licence = un appareil (Solo) ou jusqu'à 5 membres ou postes (Team). Les résultats et leur régularité dépendent aussi du modèle d'IA qui répond : un modèle plus puissant suit les étapes de sécurité plus fidèlement qu'un modèle très léger ou gratuit.

PARTIE 5 · DÉPANNAGE ET FAQ

Quand quelque chose ne va pas, décrivez-le d'abord à l'assistant en langage courant. La plupart des problèmes d'installation et d'usage quotidien se diagnostiquent directement dans l'éditeur.

✧ Essayez ceci en premier : demandez à l'IA de le corriger

Chaque fois que quelque chose ne va pas (pendant la configuration ou l'utilisation quotidienne), votre solution la plus rapide est presque toujours de **demandez à l'assistant directement dans le chat**. Décrivez ce qui s'est passé en termes simples (par exemple, « the setup stopped halfway » ou « my outil de suivi bridge won't connect ») et demandez-lui de le corriger. L'IA peut diagnostiquer et réparer la plupart des problèmes sur-le-champ. Essayez cela avant de nous écrire ; cela résout la grande majorité des problèmes en quelques secondes.

Appliquer une correction de l'IA : le bouton « Keep »

Si l'assistant corrige quelque chose en modifiant un fichier, VS Code affiche une petite barre Keep / Undo. Si vous avez demandé la correction et qu'elle vous convient, cliquez sur **Keep**, et la modification s'applique uniquement à votre propre copie, sur votre ordinateur. Cliquez sur **Undo** pour l'annuler. Rien de ce que vous gardez (Keep) n'est partagé avec qui que ce soit d'autre.

CE QUE VOUS VOYEZ	QUOI FAIRE
« Activate to start »	Saisissez Activate . Buglt ouvre le Buglt Portal dans votre navigateur. Connectez-vous, choisissez votre droit Solo ou Team, puis approuvez cet appareil. Pas encore de compte ? Obtenez - en un sur la boutique.
Le navigateur ne s'est pas ouvert, ou l'activation ne s'est pas terminée	Saisissez de nouveau Activate pour rouvrir le Buglt Portal, puis terminez la connexion et l'approbation de cet appareil. L'activation se déroule entièrement dans le navigateur ; il n'y a aucun code à coller.
« No outil de suivi enabled »	Tapez Begin setup et choisissez votre outil de suivi.
La création de ticket est refusée	Exécutez <code>python tools/connect.py jira --check</code> . Il vous dit qui vous êtes pour l'outil de suivi, ce que Buglt a le droit de créer, et énumère dans fixes le remède exact.
Il n'arrête pas de demander une activation	Saisissez Activate et terminez la connexion dans le navigateur. Buglt n'affiche jamais les valeurs des identifiants enregistrés. (Première fois ? Voir l'Étape 8c.)
Il refuse de créer un bug	Tapez Check readiness ; il liste exactement ce qui manque (généralement un jeton d'outil de suivi pas encore enregistré).
Quelque chose semble cassé après une mise à jour	Tapez Restore my settings pour revenir en arrière.
L'agent semble « sortir du script »	Tapez Read and follow all your given instructions , ou démarrez un nouveau chat. Cela peut arriver plus souvent avec un modèle d'IA léger/gratuit ; un modèle plus puissant dans le sélecteur de modèle de Copilot est plus constant.
Les réponses semblent génériques / pas spécifiques au projet	Donnez - lui plus de contexte, ou montrez - lui un ticket existant pendant Begin setup pour qu'il apprenne votre style. Relancez Begin setup pour affiner.
Vous voulez changer un choix de configuration	Tapez Begin setup et dites ce qu'il faut changer (par ex. « add a outil de suivi » ou « start over ») et seule cette partie se rouvre. Si la configuration a simplement été interrompue en cours de route, un simple Begin setup reprend là où elle s'était arrêtée plutôt que de tout recommencer.

Vérifications de santé intégrées

`Check status` (mes outils sont-ils connectés ?) · `Check readiness` (que reste-t-il avant de pouvoir créer un ticket ?). Pour une vérification plus poussée, ouvrez Terminal → New Terminal et lancez `python tools/selftest.py`.

Foire aux questions

Dois-je savoir coder ?

Non. Si vous savez installer une application et taper une phrase, vous pouvez utiliser Buglt. Il s'occupe des aspects techniques pour vous.

Va-t-il créer des bugs sans que je le sache ?

Jamais. Chaque ticket et commentaire est présenté en aperçu ; les actions irréversibles exigent de saisir `FILE IT`, car un simple « oui » ne suffit pas. Utilisez `dry run` pour vous entraîner : tout ticket, commentaire, modification, pièce jointe, suppression et notification externes sont supprimés (les commandes locales que vous lancez vous-même, comme la sauvegarde ou l'export, peuvent encore créer des fichiers locaux) et l'agent refuse les écritures dans l'outil de suivi (ce refus suit les instructions de Buglt, ce n'est pas un verrou de plateforme ; utilisez des identifiants en lecture seule pour évaluer).

Dois-je démarrer un serveur à chaque fois ?

Non. La création de tickets utilise le jeton d'API que vous avez enregistré avec `python tools/connect.py` : il n'y a donc rien à démarrer. Un serveur MCP n'est nécessaire que pour les intégrations facultatives en lecture seule, comme Confluence, Sentry et Notion.

Puis-je l'utiliser sur deux ordinateurs ?

Un appareil à la fois en Solo (jusqu'à 5 membres en Équipe, un par membre). Passer à un nouvel appareil ne pose aucun problème : activez le nouveau, et l'ancien ne peut plus renouveler immédiatement. Sa place se libère dès que l'ancien appareil se reconnecte (confirmant qu'il a cédé l'accès) ou que sa licence hors ligne de 72 heures expire, il n'y a donc aucun délai d'attente fixe. Tapez `License status` pour vérifier.

Et si le serveur de licence est en panne ?

Vous continuez à travailler. Après une activation en ligne réussie, une licence en cache vous permet de travailler hors ligne jusqu'à 72 heures, si bien qu'une panne (ou simplement un week-end sans internet) ne vous interrompt pas pendant cette fenêtre. Passé ce délai, une nouvelle vérification en ligne est requise.

Ai-je besoin de GitHub Copilot ?

C'est la solution la plus simple. Si vous ne l'avez pas, lancez `python tools/setup_wizard.py` dans le terminal pour configurer, et Buglt peut fonctionner en mode autonome avec votre propre clé OpenAI/Anthropic (`python standalone/run.py`).

Mes données sont-elles privées ?

Oui. Taskivator ne reçoit que les données de licence/mise à jour : la connexion à votre compte Buglt (dans le navigateur, sur le Portal), un identifiant d'appareil haché à sens unique et le droit Solo ou Team que vous approuvez pour cet appareil. Le logiciel Buglt n'utilise pas la mesure Google Ads et n'envoie aucune télémétrie produit. Il n'y a aucun code de licence, et votre mot de passe reste dans le navigateur. Vos tickets eux-mêmes ne vont qu'au modèle d'IA et à l'outil de suivi que vous connectez, jamais à nous.

Puis-je modifier les fichiers de BugIt ?

Vous êtes censé personnaliser vos propres éléments : votre glossaire, votre style maison et vos paramètres (Partie 3). Ne désactivez simplement pas la licence/l'activation. Si vous trouvez un vrai bug dans l'outil lui-même, écrivez au support pour qu'il soit corrigé pour tout le monde dans la prochaine mise à jour.

Avec quels outils de suivi fonctionne-t-il ?

Onze, et BugIt crée des tickets dans chacun d'eux : Jira Cloud, Azure DevOps, GitHub Issues, GitLab Issues, Bugzilla, YouTrack, Linear, Shortcut, ClickUp, Asana et Trello. Chacun a une configuration guidée, chacun écrit avec un identifiant que vous créez dans votre propre compte, et BugIt le vérifie face à votre destination avant de l'enregistrer. Ce qui change, ce sont les à-côtés : GitHub, GitLab et Bugzilla n'ont aucun envoi de pièce jointe, ClickUp accepte un fichier sans pouvoir le retirer, et GitHub, GitLab et Trello n'ont pas de champ de priorité, la gravité y est donc écrite dans le texte du ticket. Choisissez le vôtre pendant la configuration et changez à tout moment avec [Begin setup](#).

Détectera-t-il les bugs en double avant que je le crée ?

Oui, pour un rapport standard. Avant que quoi que ce soit ne soit écrit, il recherche dans votre outil de suivi des rapports similaires (même formulés un peu différemment) et vous montre les correspondances, pour que vous puissiez éviter de soulever le même bug deux fois. C'est toujours vous qui décidez de continuer ou non. Le formulaire rapide saute cette recherche exprès, pour éviter l'aller-retour, et son aperçu vous le dit.

Peut-il rédiger des tickets dans plusieurs langues ?

Oui. Ajoutez une deuxième langue pendant la configuration et chaque rapport est rédigé dans les deux, conservées dans des blocs séparés, en utilisant votre glossaire pour que les termes du produit restent exacts ; il n'improvise jamais une traduction.

Puis-je joindre une capture d'écran ?

Demandez à BugIt de les joindre. Il crée un dossier de preuves, l'ouvre et vous en donne le chemin exact ; déposez-y vos fichiers puis dites que vous avez terminé. BugIt supprime alors tout ce qu'il remarque de sensible, vous montre un aperçu et les joint après son propre [FILE IT](#). BugIt ne peut pas atteindre un fichier collé dans le chat, c'est pourquoi il vous propose toujours le dossier.

Masque-t-il les mots de passe et jetons dans mes rapports ?

Quand le masquage est activé, il supprime les e-mails, jetons et adresses IP de chaque brouillon avant la création. Vous voyez toujours le texte complet dans votre aperçu au préalable.

J'ai créé un ticket par erreur. Puis-je l'annuler ?

L'aperçu et la confirmation explicite (saisir [FILE IT](#)) évitent la plupart des erreurs avant même qu'elles ne soient écrites. Si vous avez activé le journal d'audit, tapez [Undo last filing](#). Sinon, fermez ou modifiez simplement le ticket dans votre outil de suivi comme d'habitude.

Peut-il m'aider à vérifier ou clôturer un bug ?

Oui. Demandez un commentaire de vérification et il rédige une note claire de réussite/échec (dans vos langues) sur le ticket existant, et vous l'approuvez avant qu'il ne soit publié.

Puis-je l'utiliser pour plusieurs projets ?

Chaque dossier BugIt est configuré pour l'outil de suivi d'un seul projet. Pour un autre projet, relancez [Begin setup](#) pour le rediriger ailleurs, ou conservez une copie distincte par projet.

Comment obtenir les mises à jour ?

Quand une nouvelle version sort, BugIt le mentionne quand vous démarrez un chat. Tapez **update** et il s'installe sur place : votre glossaire, votre style maison et vos paramètres sont conservés, et il les sauvegarde d'abord.

Que se passe-t-il quand ma licence expire ?

À la fin de votre période, renouvelez depuis votre compte ; votre appareil applique le droit renouvelé lors de sa prochaine vérification en ligne. La licence hors ligne de 72 heures mentionnée plus haut est distincte ; elle ne couvre qu'une panne temporaire du *serveur* de licence, pas une période expirée. Tapez **License status** à tout moment.

Si je renouvelle ou j'achète une autre licence, les jours s'additionnent-ils ?

Non, les licences ne s'additionnent pas. Un renouvellement **remplace** votre droit actuel ; sa durée redémarre à zéro et les jours inutilisés ne sont pas reportés. Activez donc un renouvellement vers la fin de votre période actuelle, pas trop tôt.

PARTIE 6 · LICENCE ET ASSISTANCE

Conditions de licence (résumé en langage clair)

Les conditions complètes et contraignantes figurent dans le fichier **LICENSE** de votre dossier BugIt, et c'est ce document qui fait foi. En résumé :

SUJET	EN CLAIR
Licence, pas vente	Vous achetez une licence d'utilisation de BugIt, pas la propriété. Taskivator conserve tous les droits.
Vos postes	Solo = un appareil à la fois ; Équipe = jusqu'à 5 membres. Votre compte et votre droit vous sont personnels ; ne les partagez pas, ne les revendez pas, ne les publiez pas.
Révocation	Un accès partagé, remboursé, contesté (chargeback) ou utilisé à mauvais escient peut être révoqué.
Personnaliser, mais...	Modifiez librement votre config, votre glossaire et vos modèles, mais ne désactivez pas et ne contournez pas la licence/l'activation.
Votre responsabilité	La façon dont vous utilisez BugIt et la sécurité de votre projet vous appartiennent. Vous relisez et approuvez chaque rapport avant sa création. Les protections sont des aides, pas des garanties.
« Tel quel », responsabilité	Fourni tel quel, sans garantie. Dans la mesure permise par la loi, la responsabilité est plafonnée au montant payé, à l'exclusion des dommages indirects/consécutifs.
Durée et droit applicable	Une licence d'un an débutant le jour de l'activation (un achat unique, sans reconduction automatique), régie par le droit japonais. Les remboursements sont gérés par la boutique où vous avez acheté.

Les renouvellements ne s'additionnent pas

Un renouvellement remplace votre droit actuel : sa durée redémarre à zéro et les jours inutilisés ne sont pas reportés, renouvelez donc vers la fin de votre période.

Deux documents dans votre dossier

[LICENSE](#) (conditions complètes) et [PRIVACY.md](#) (déclaration de confidentialité). En activant et en utilisant BugIt, vous acceptez la LICENSE.

Obtenir de l'aide

Merci de consulter d'abord le Dépannage et la FAQ ci-dessus ; ils couvrent presque tout. Ensuite, dans cet ordre :

1 · Demandez à l'IA de le corriger

Votre meilleur premier réflexe : décrivez le problème dans le chat et demandez à l'assistant de le corriger. S'il modifie un fichier et que ça semble correct, cliquez sur Keep pour l'appliquer (uniquement sur votre PC).

2 · Lancez une vérification de santé

[Check status](#) · [Check readiness](#) · ou [python tools/selftest.py](#) dans le terminal.

3 · Restaurez

[Restore my settings](#) annule une mauvaise modification ou mise à jour.

4 · Contactez un humain

Seulement si le guide et l'IA n'ont pas pu vous aider : visitez [bugit.dev](#) ou écrivez à support@bugit.dev (support assuré en anglais). Souci de configuration dans vos 7 premiers jours ? Nous vous aiderons à démarrer, ou à obtenir un remboursement via la boutique.

Merci de ne pas inclure d'informations confidentielles sur votre projet dans vos e-mails au support

Chaque projet est différent, et une grande partie de votre travail peut être confidentielle. Si vous nous écrivez, merci de décrire le problème BugIt en termes généraux uniquement, et ne collez pas de code confidentiel, de détails de bugs, de spécifications, de captures d'écran ou de données de votre projet. Taskivator n'est pas responsable des informations confidentielles qui nous sont envoyées, et tout élément confidentiel que nous recevons est supprimé immédiatement. Dans la mesure du possible, demandez d'abord à l'assistant IA de vous aider ; il peut résoudre la plupart des problèmes directement dans votre éditeur.

Merci d'avoir choisi BugIt.

Créez des tickets propres, évitez les doublons, et récupérez votre temps, un bug à la fois.

BugIt QA Agent · Guide de configuration et d'utilisation · © 2026 Taskivator. All Rights Reserved. · [bugit.dev](#) · Les fichiers LICENSE et PRIVACY.md inclus avec votre téléchargement sont les documents qui font foi.